

Python语言与系统设计

史骏

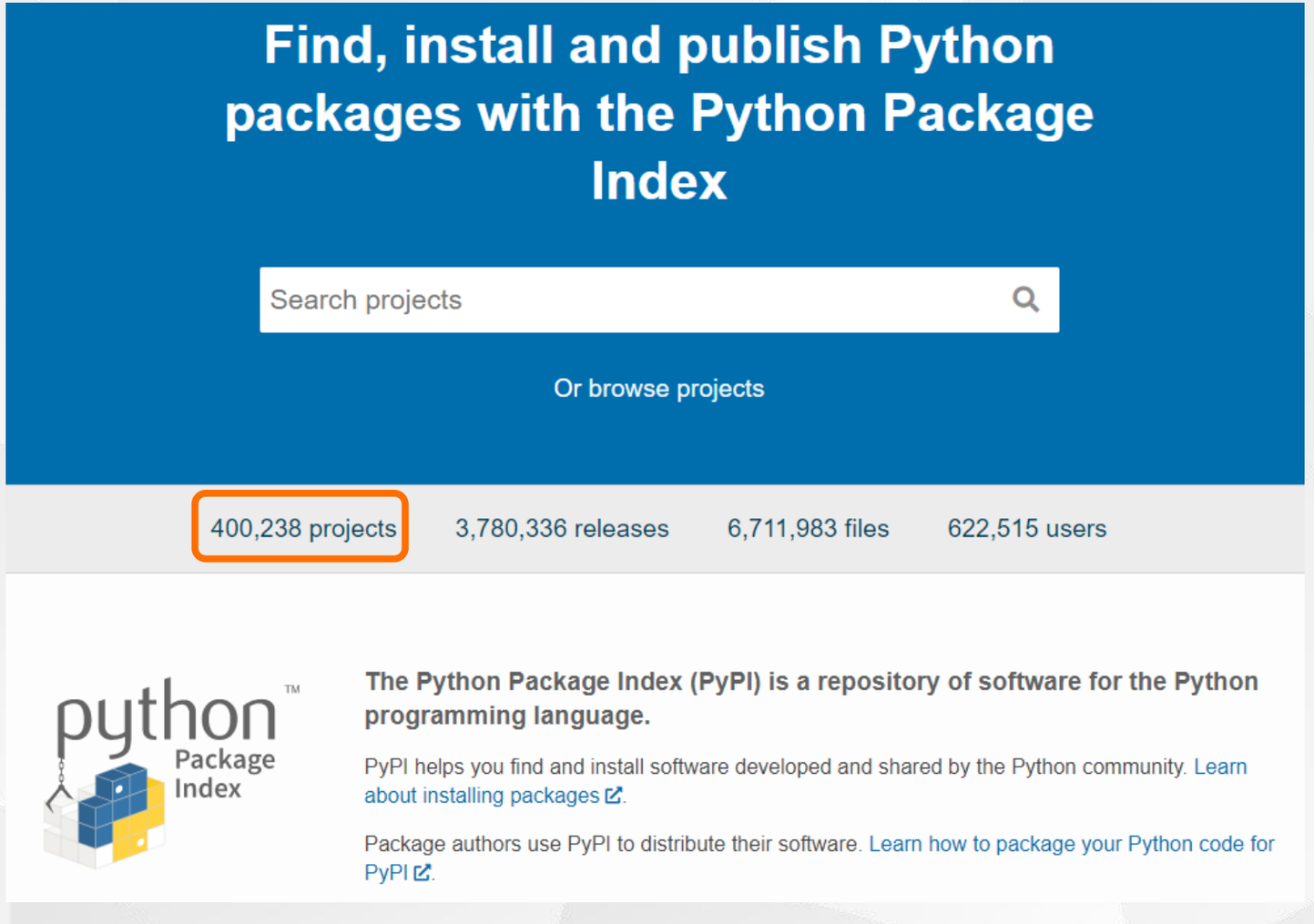
juns@hfut.edu.cn

计算思维

- 逻辑思维：推理和演绎，数学为代表， A->B B->C A->C
- 实证思维：实验和验证，物理为代表，引力波<-实验
- 计算思维：设计和构造，计算机为代表，汉诺塔递归

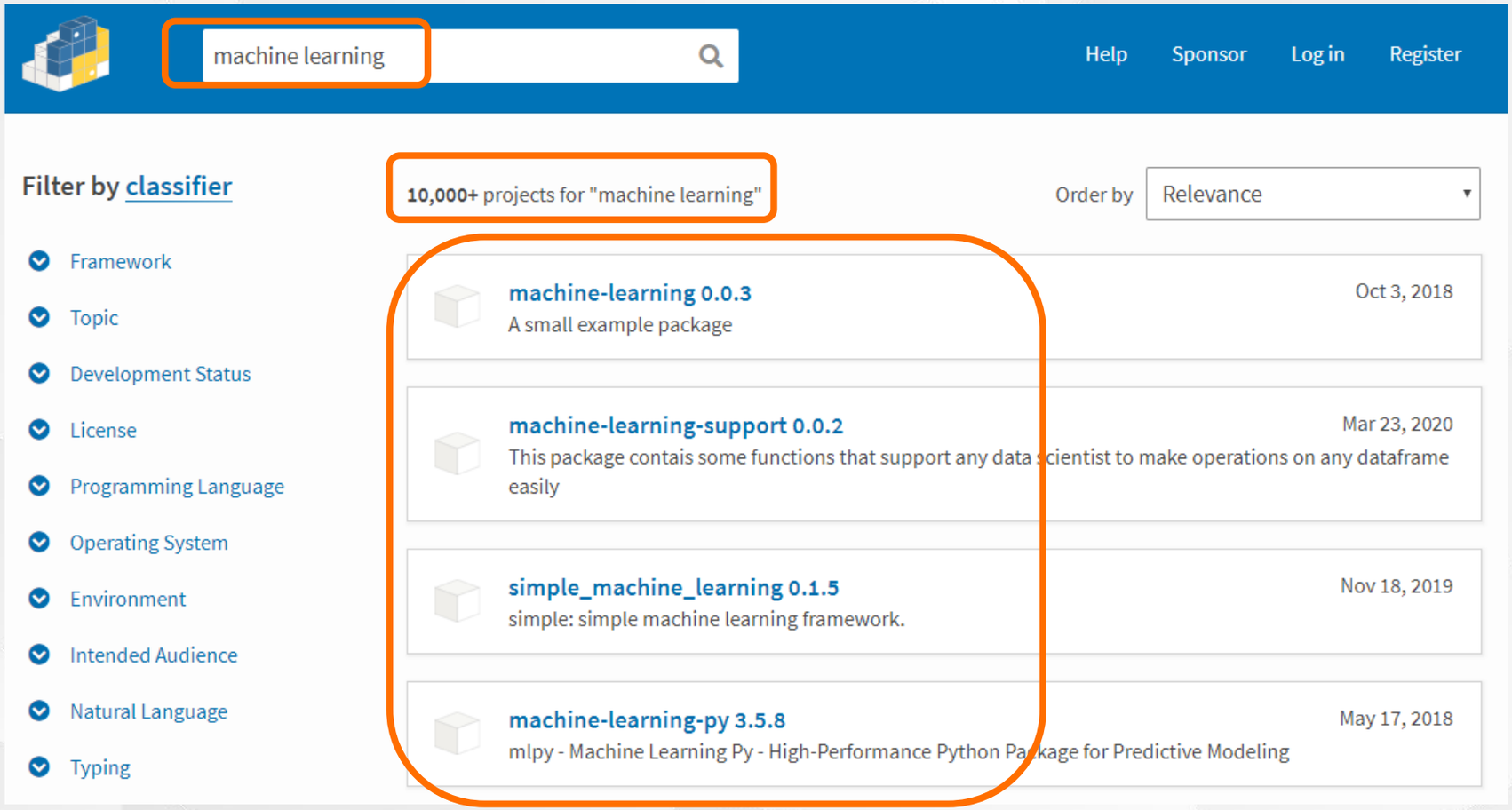
Python社区

>40万个第三方库 https://pypi.org/



Python社区

实例：开发与机器学习相关的程序



安装Python第三方库

三种方法

- 方法1(主要方法): 使用pip命令
- 方法2: 集成安装方法
- 方法3: 文件安装方法

pip安装方法

使用pip安装工具（命令行下执行）

```
C:\Users\Lenovo>pip -h

Usage:
  pip <command> [options]

Commands:
  install                     Install packages.
  download                    Download packages.
  uninstall                   Uninstall packages.
  freeze                      Output installed packages in requirements format.
  list                        List installed packages.
  show                        Show information about installed packages.
  check                       Verify installed packages have compatible dependencies.
  config                      Manage local and global configuration.
  search                      Search PyPI for packages.
  wheel                       Build wheels from your requirements.
  hash                        Compute hashes of package archives.
  completion                  A helper command used for command completion.
  help                        Show help for commands.
```

pip安装方法

常用的pip命令

```
D:\>pip install <第三方库名>
```

安装指定的第三方库

pip安装方法

常用的pip命令

```
D:\>pip uninstall <第三方库名>
```

卸载指定的第三方库

pip安装方法

常用的pip命令

```
D:\>pip download <第三方库名>
```

下载但不安装指定的第三方库

pip安装方法

常用的pip命令

```
D:\>pip show <第三方库名>
```

列出某个指定第三方库的详细信息

pip安装方法

常用的pip命令

```
D:\>pip search <关键词>
```

根据关键词在名称和介绍中搜索第三方库

pip安装方法

常用的pip命令

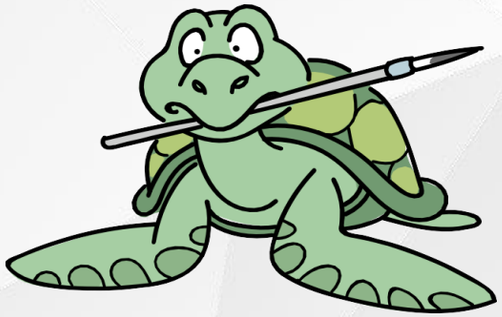
```
D:\>pip list
```

列出当前系统已经安装的第三方库

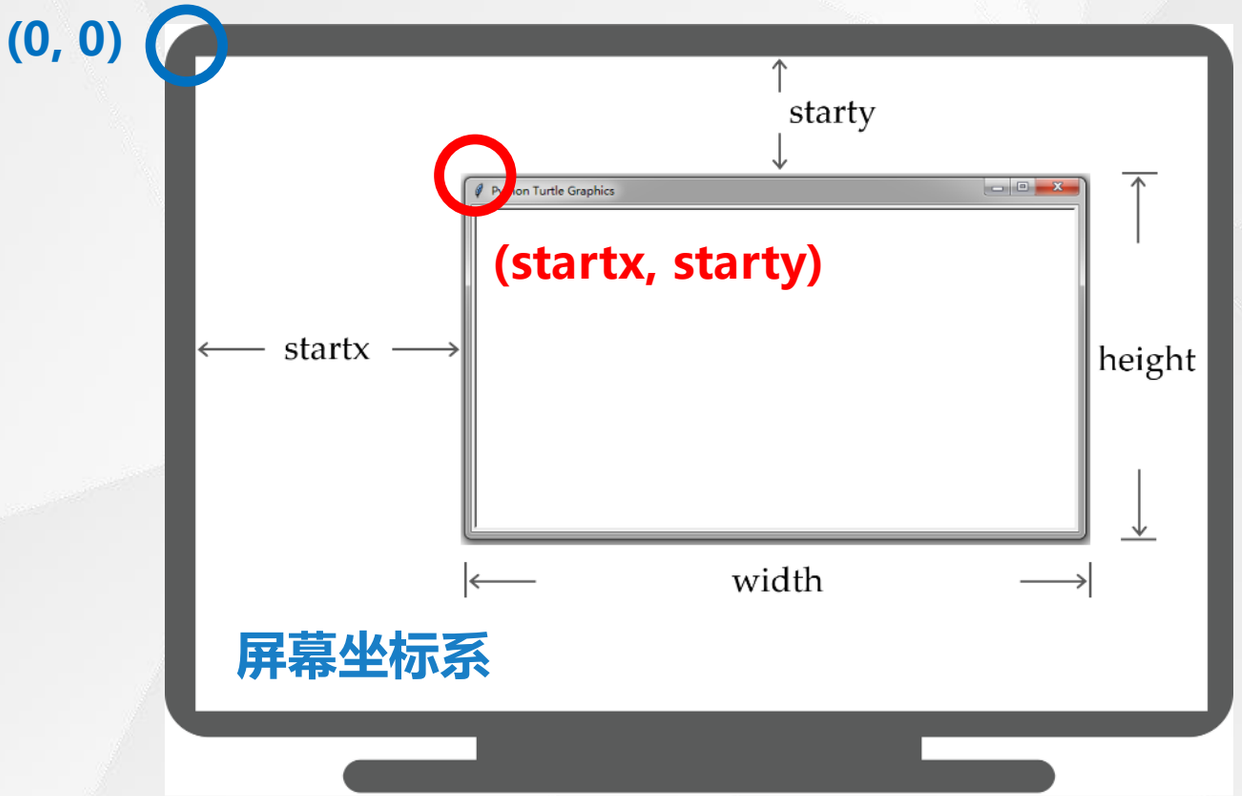
turtle库

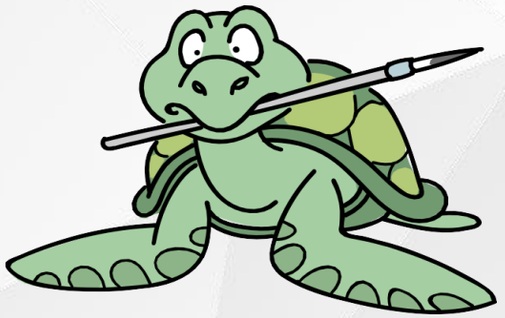
python入门级的图形绘制函数库

- 有一只海龟，其实在窗体正中心，在画布上游走
- 走过的轨迹形成了绘制的图形
- 海龟由程序控制，可以变换颜色、改变宽度等



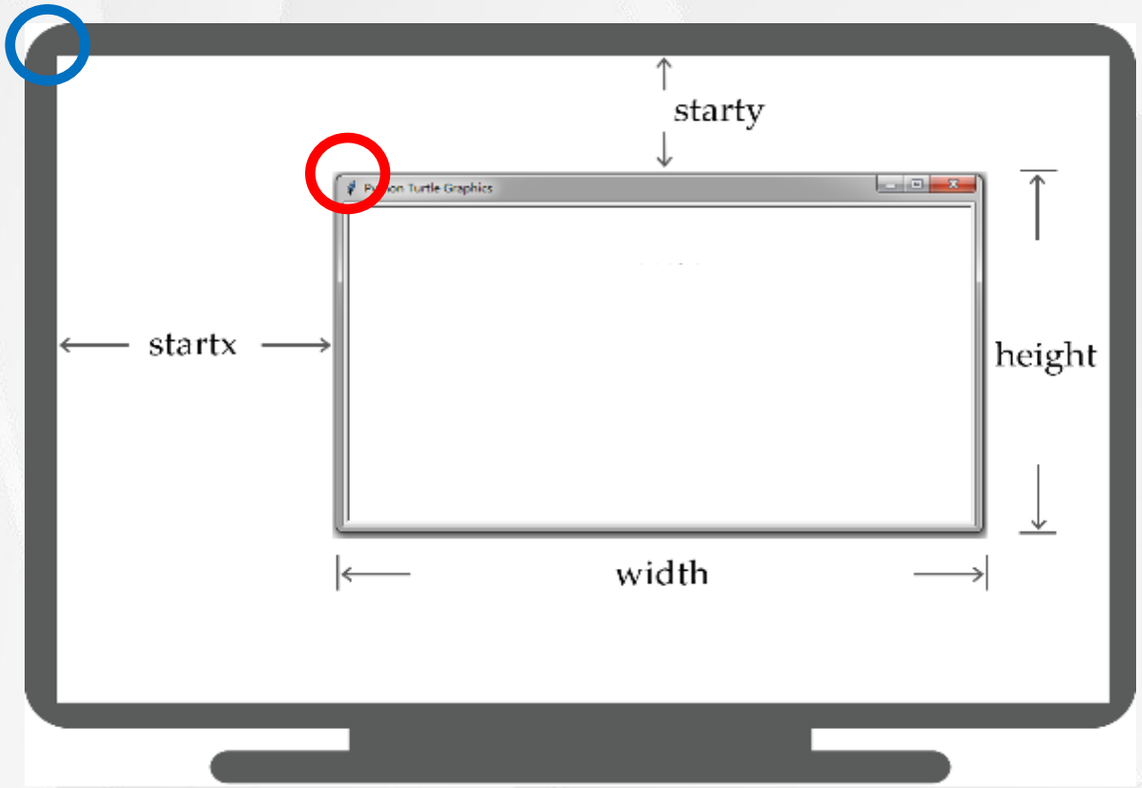
turtle的绘图窗体



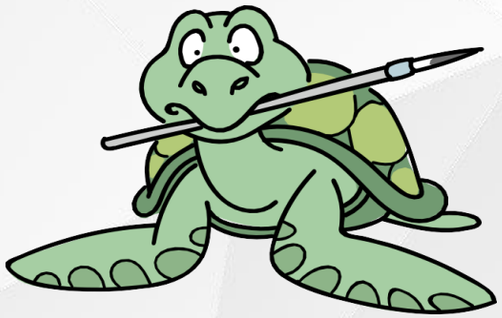


turtle的绘图窗体

```
turtle.setup(width, height, startx, starty)
```

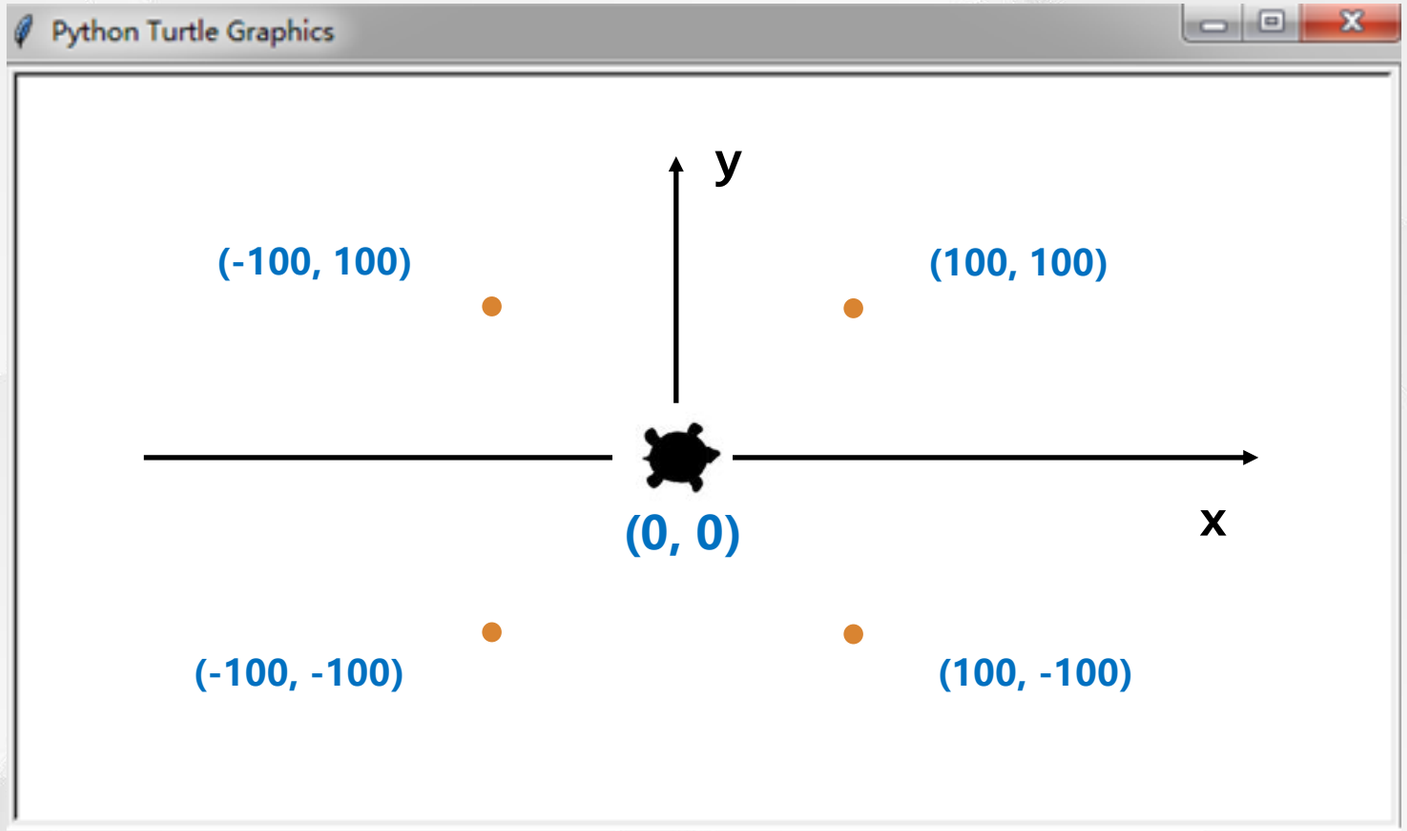


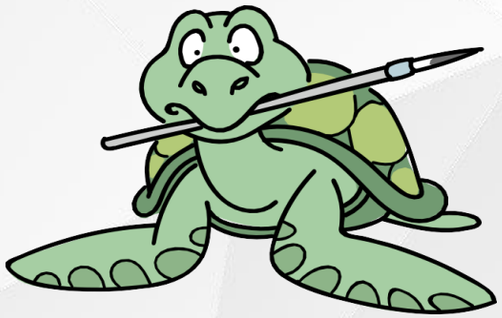
- setup()设置窗体大小及位置
- 4个参数中后两个可选
- setup()不是必须的



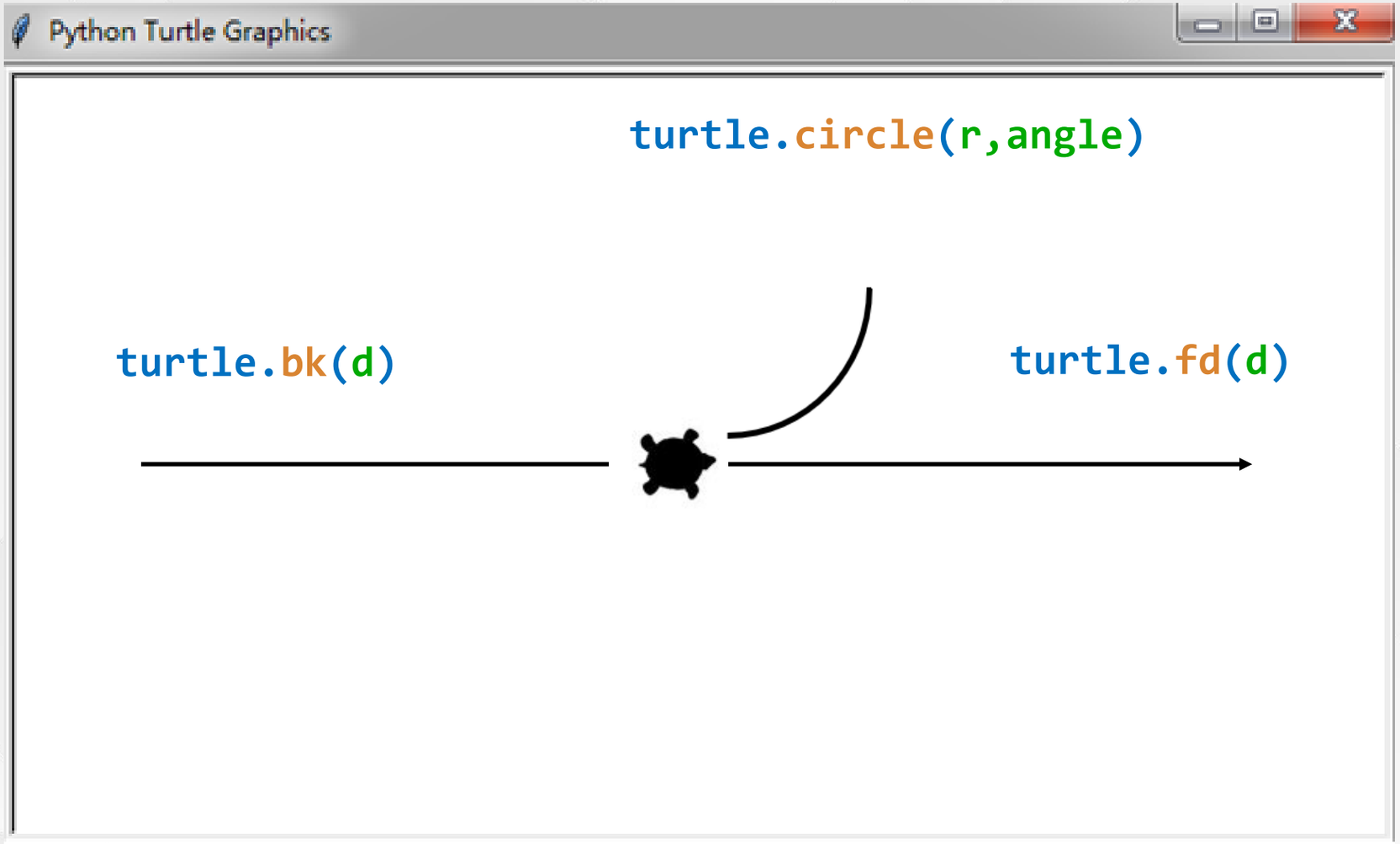
turtle的空间坐标体系

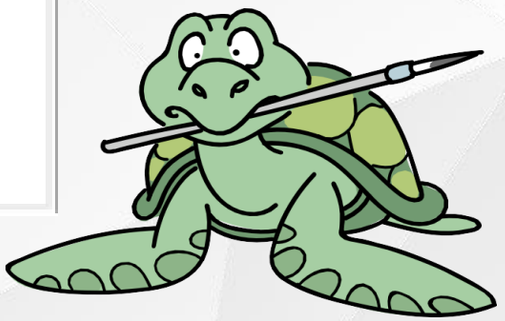
```
turtle.goto(x, y)
```





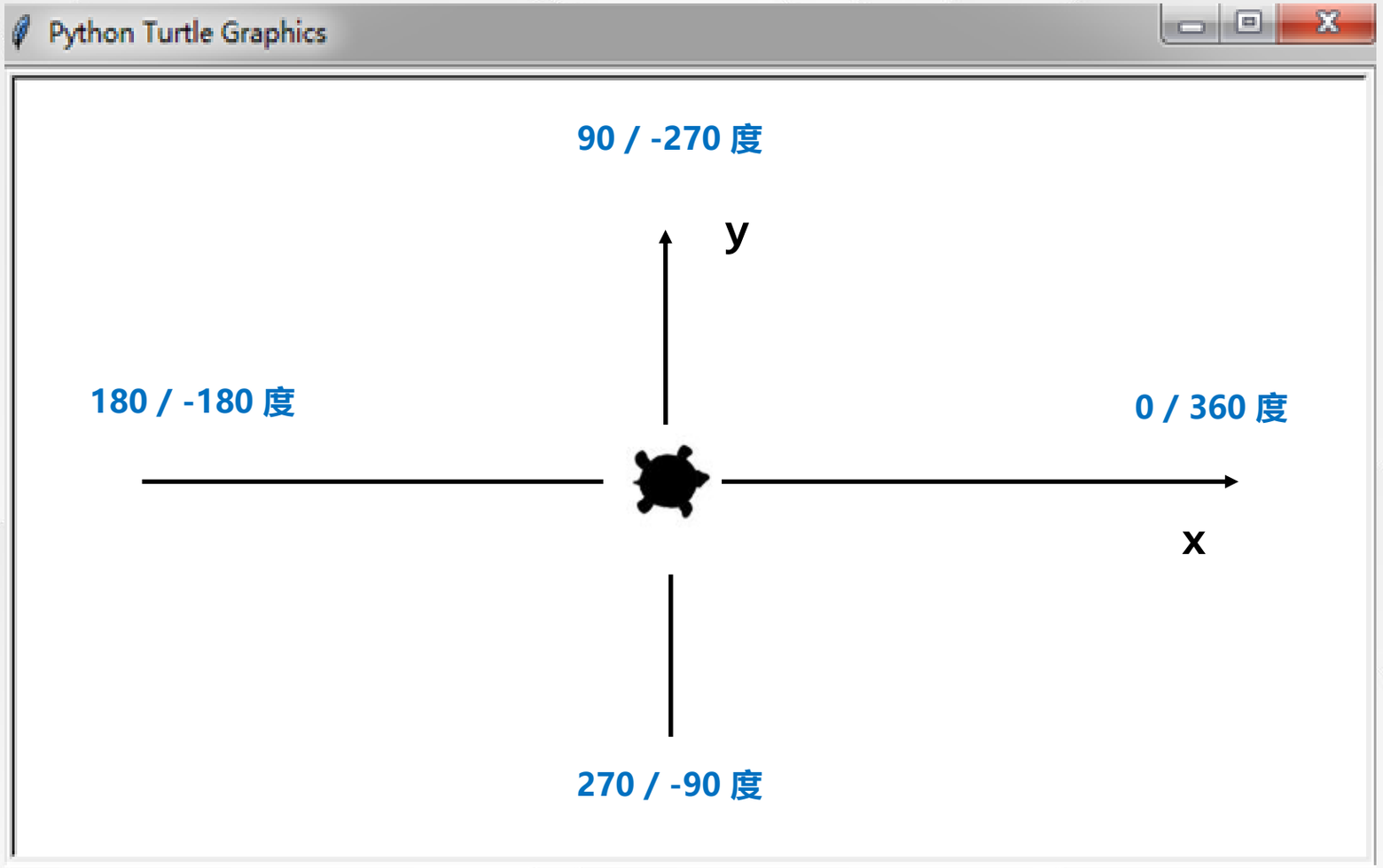
turtle的空间坐标体系

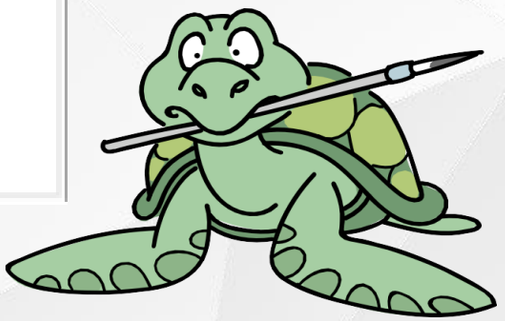




turtle的角度坐标体系

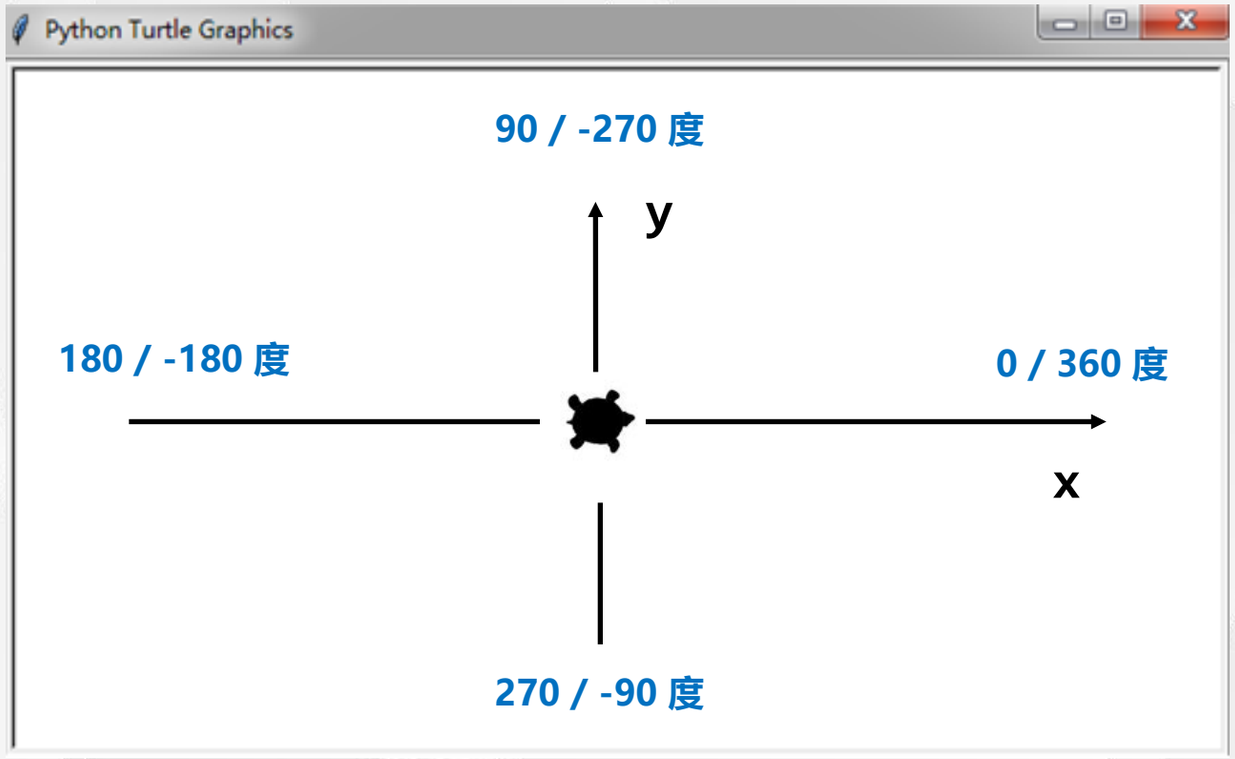
绝对角度



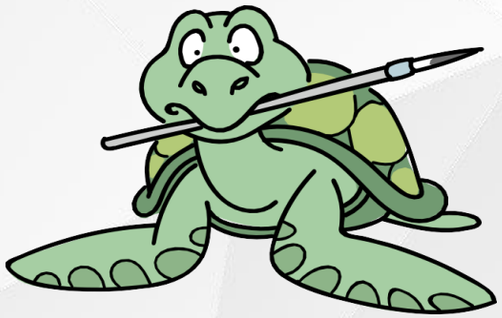


turtle的角度坐标体系

```
turtle.seth(angle)
```

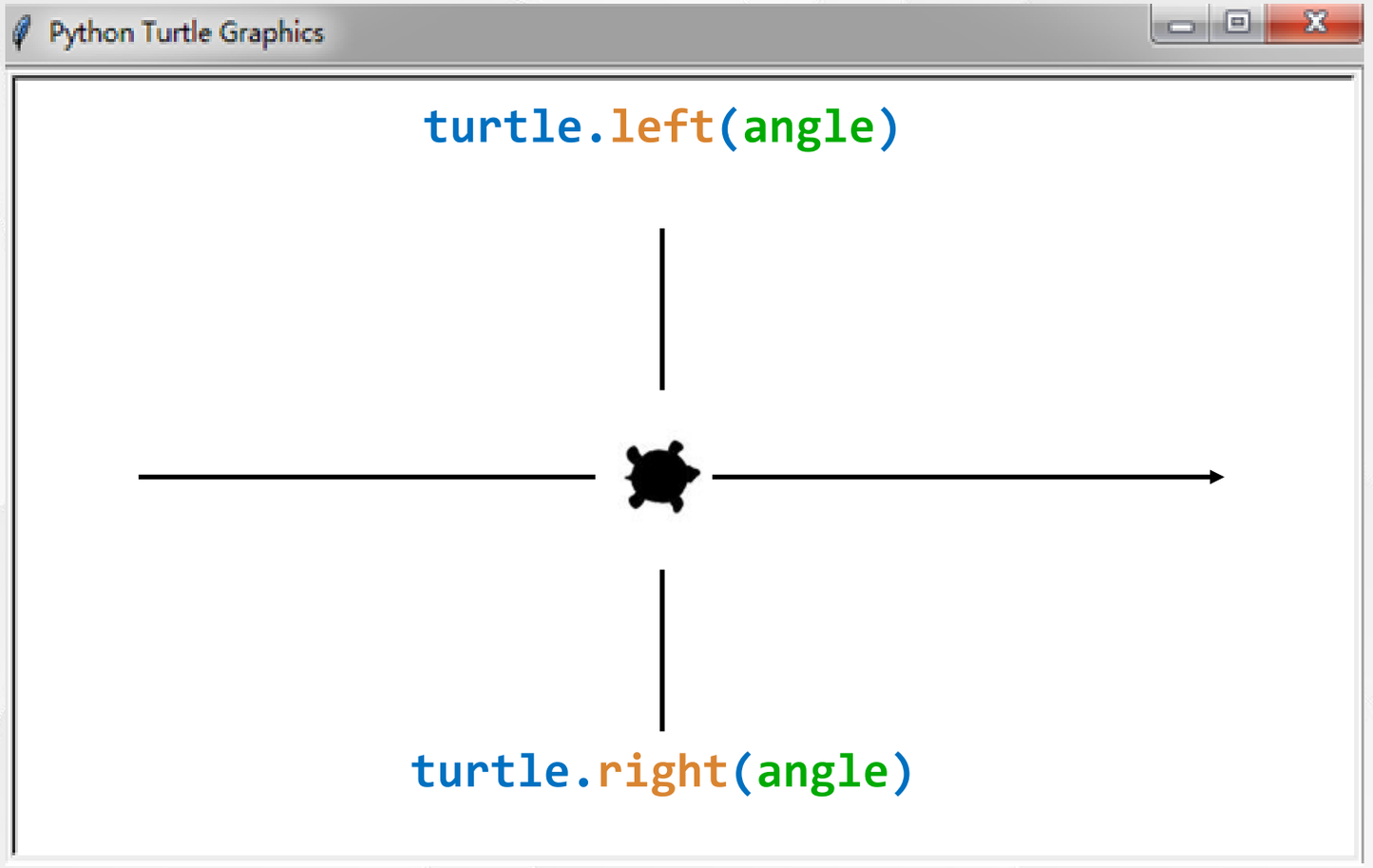


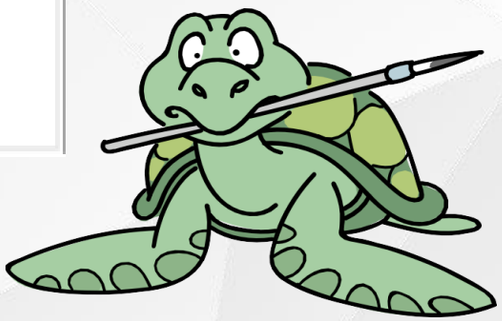
- seth()改变海龟行进方向
- angle为绝对度数
- seth()只改变方向但不行进



turtle的角度坐标体系

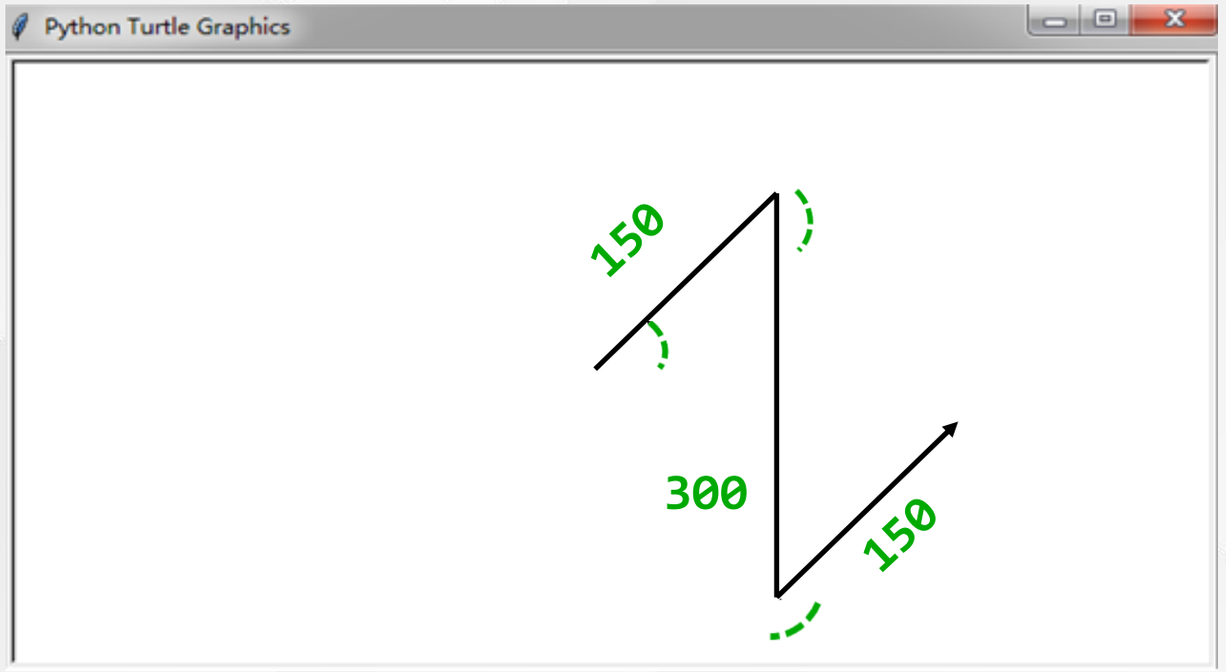
海龟角度

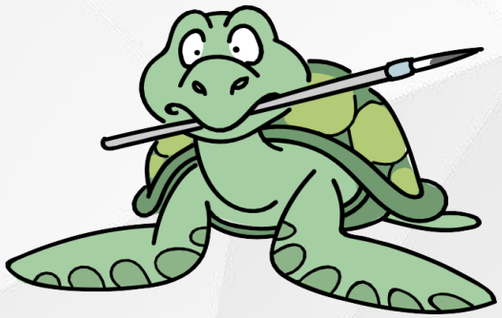




turtle的角度坐标体系

```
import turtle
turtle.left(45)
turtle.fd(150)
turtle.right(135)
turtle.fd(300)
turtle.left(135)
turtle.fd(150)
```





RGB色彩模式

由三种颜色构成的万物色



- RGB指红蓝绿三个通道的颜色组合
- 覆盖视力所能感知的所有颜色
- RGB每色取值范围0-255整数或0-1小数

RGB色彩模式

英文名称	RGB整数值	RGB小数值	中文名称
white	255, 255, 255	1, 1, 1	白色
yellow	255, 255, 0	1, 1, 0	黄色
magenta	255, 0, 255	1, 0, 1	洋红
cyan	0, 255, 255	0, 1, 1	青色
blue	0, 0, 255	0, 0, 1	蓝色
black	0, 0, 0	0, 0, 0	黑色

RGB色彩模式

英文名称	RGB整数值	RGB小数值	中文名称
seashell	255, 245, 238	1, 0.96, 0.93	海贝色
gold	255, 215, 0	1, 0.84, 0	金色
pink	255, 192, 203	1, 0.75, 0.80	粉红色
brown	165, 42, 42	0.65, 0.16, 0.16	棕色
purple	160, 32, 240	0.63, 0.13, 0.94	紫色
tomato	255, 99, 71	1, 0.39, 0.28	番茄色

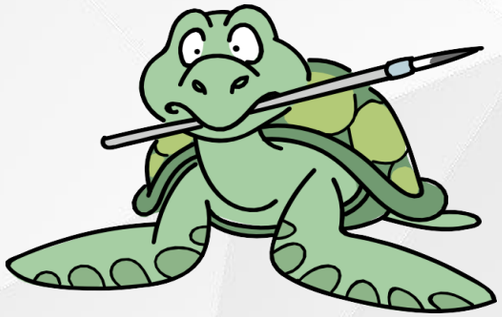
turtle的RGB色彩模式

默认采用小数值 可切换为整数值



```
turtle.colormode(mode)
```

- 1.0：RGB小数值模式
- 255：RGB整数值模式



turtle的画笔控制函数

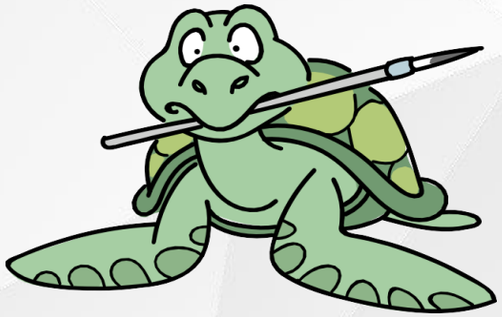
画笔操作后一直有效，一般成对出现

- turtle.penup()　　别名　　turtle.pu()

抬起画笔，海龟在飞行

- turtle.pendown()　　别名　　turtle.pd()

落下画笔，海龟在爬行



turtle的画笔控制函数

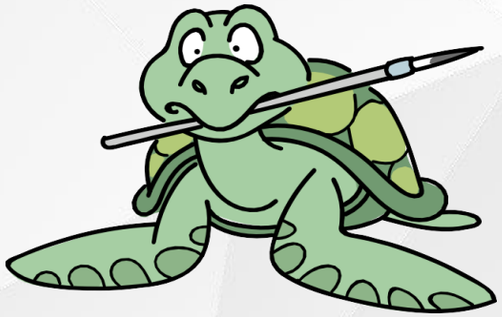
画笔设置后一直有效，直至下次重新设置

- turtle.pensize(width)　别名　turtle.width(width)

  画笔宽度，海龟的腰围

- turtle.pencolor(color)　color为颜色字符串或r,g,b值

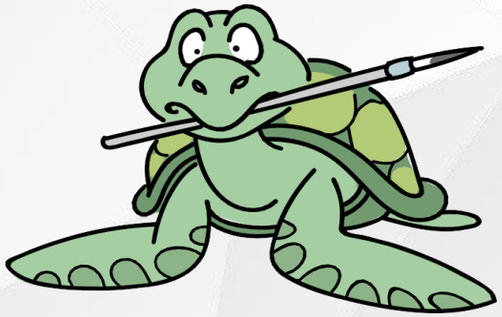
  画笔颜色，海龟在涂装



turtle的画笔控制函数

pencolor(color)的color可以有三种形式

- 颜色字符串　：turtle.pencolor("purple")
- RGB的小数值：turtle.pencolor(0.63, 0.13, 0.94)
- RGB的元组值：turtle.pencolor((0.63,0.13,0.94))



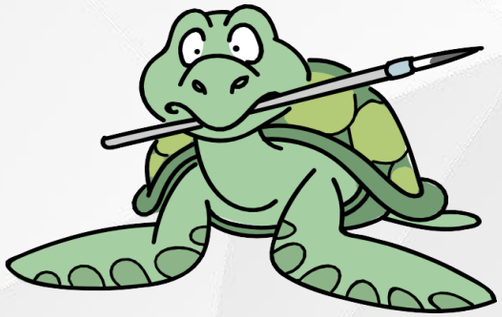
turtle的运动控制函数

控制海龟行进：走直线 & 走曲线

- turtle.forward(d)　别名　turtle.fd(d)

向前行进，海龟走直线

- d: 行进距离，可以为负数



turtle的运动控制函数

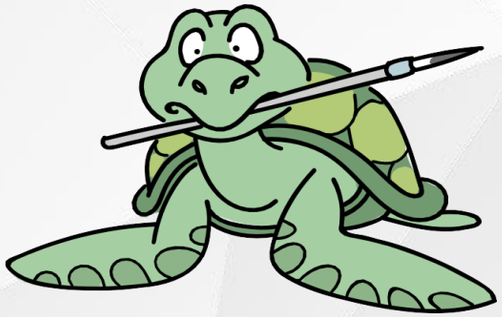
控制海龟行进：走直线 & 走曲线

- turtle.circle(r, extent=None)

  根据半径r绘制extent角度的弧形

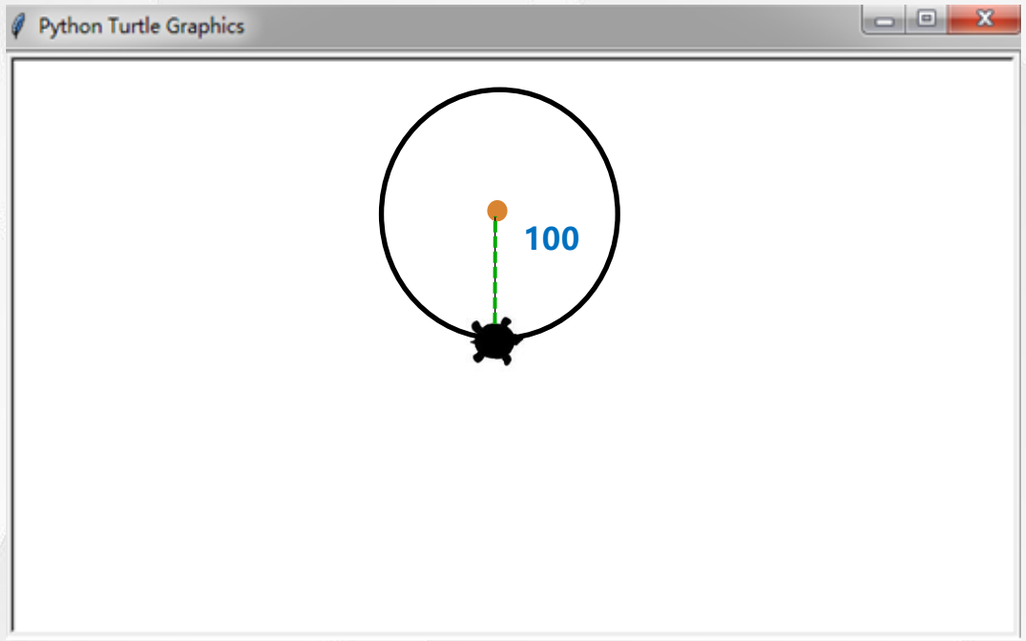
- r: 默认圆心在海龟左侧r距离的位置

- extent: 绘制角度，默认是360度整圆

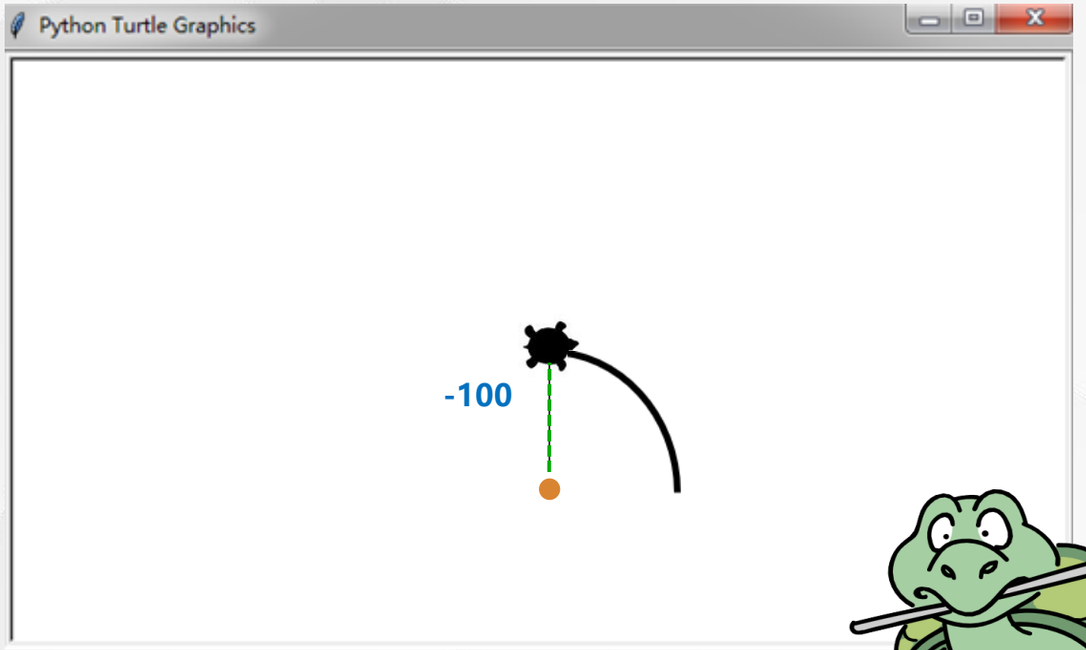


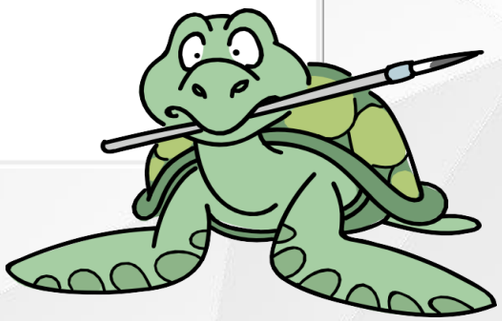
turtle的运动控制函数

```
turtle.circle(100)
```




```
turtle.circle(-100,90)
```





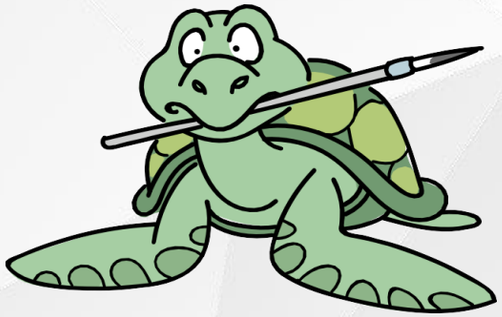
turtle的方向控制函数

控制海龟面对方向: 绝对角度 & 海龟角度

- turtle.setheading(angle)　　别名　　turtle.seth(angle)

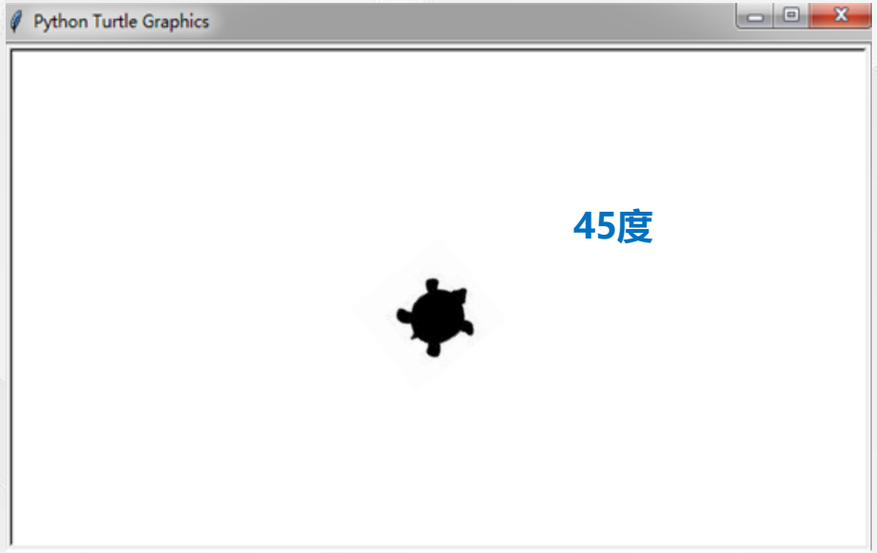
  改变行进方向，海龟走角度

- angle: 行进方向的绝对角度

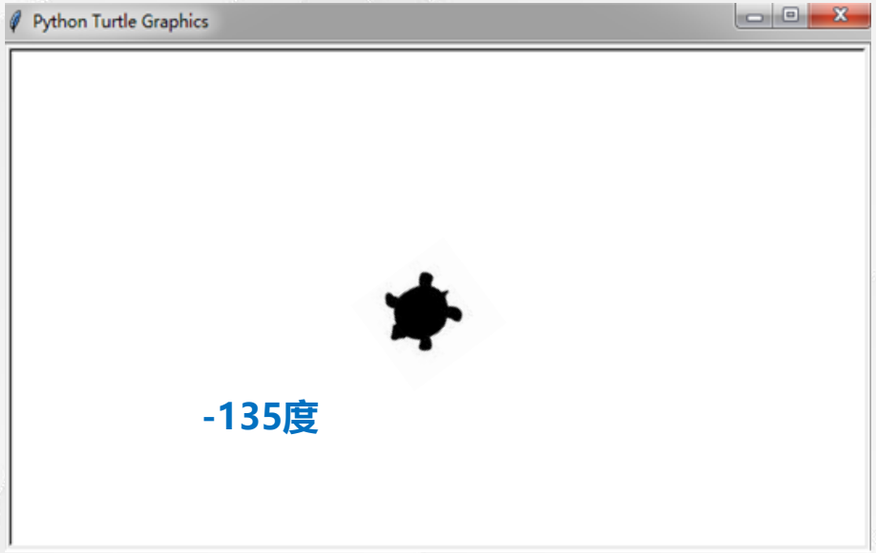


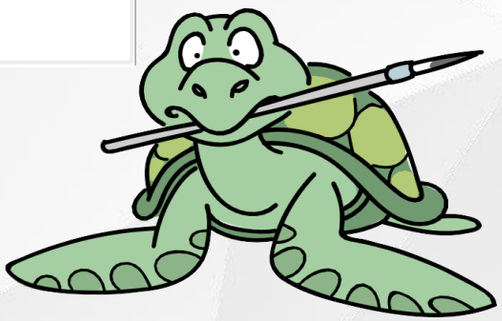
turtle的方向控制函数

```
turtle.seth(45)
```




```
turtle.seth(-135)
```

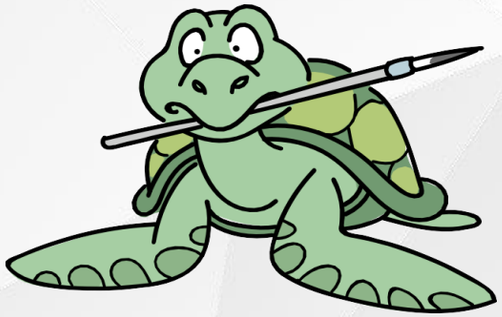




turtle的方向控制函数

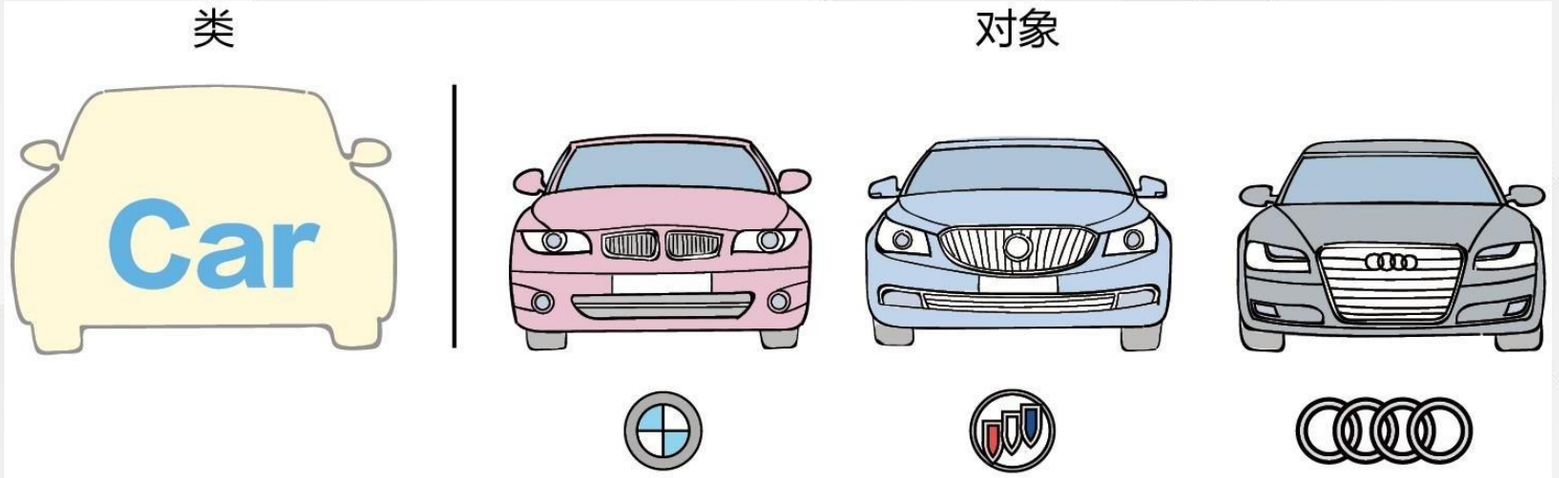
控制海龟面对方向: 绝对角度 & 海龟角度

- turtle.left(angle)　　海龟向左转
- turtle.right(angle)　　海龟向右转
- angle: 在海龟当前行进方向上旋转的角度



面向对象程序设计

面向对象是一种编程思想，即按照真实世界的思维方式构建软件系统



类的定义

类的定义以关class开始，类名后面紧跟冒号":"

```
class Person: # Python的类名约定以大写字母开头
  #----类的一个示例----
  # 属性
  skincolor = "yellow"
  high = 168
  weight = 65
  # 方法
  def goroad(self):
        print("人走路动作的测试……")
  def sleep(self):
        print("睡觉，晚安！")
```

- 属性就是变量，静态的特征
- 方法就是一个个的函数，通过这些函数来描述动作行为
- 类名约定以大写字母开头，函数约定以小写字母开头

类的使用

类定义好之后，就可将类实例化为对象

对象名 = 类名()

```
class Person: # Python的类名约定以大写字母开头
  #----类的一个示例----
  # 属性
  skincolor = "yellow"
  high = 168
  weight = 65
  # 方法
  def goroad(self):
        print("人走路动作的测试……")
  def sleep(self):
        print("睡觉，晚安！")
```

```
p = Person()
```

将类Person实例化为对象p

类的构造方法

类的构造方法：__init__(self)

- 只要实例化一个对象的时候，这个方法就会在对象被创建的时候自动调用
- 实例化对象的时候是可以传入参数的，这些参数会自动传入

```
>>> class Bear:
    def __init__(self,name):
        self.name = name
    def kill(self):
        print("%s,是保护动物，不能杀..."% self.name)


>>> a=Bear('狗熊')
>>> a.kill()
狗熊,是保护动物，不能杀...
```

类的访问权限

- C++/java：通过关键字public、private来表明访问的权限是公有、私有的
- Python：默认情况下对象的属性和方法都是公开的，公有的，通过点(.)操作符来访问

```
>>> class Bear:
    name = "树袋熊"


>>> b=Bear()
>>> b.name
'树袋熊'
```

类的属性与方法

- 类的私有属性

__private_attrs：两个下划线开头，声明该属性为私有，不能在类的外部被使用或直接访问。在类内部的方法中使用时 self.__private_attrs

- 类的方法

在类的内部，使用 def 关键字来定义一个方法，与一般函数定义不同，类方法必须包含参数 self，且为第一个参数，self 代表的是类的实例。

self 的名字并不是规定死的，也可以使用 this，但是最好还是按照约定是用 self

- 类的私有方法

__private_method：两个下划线开头，声明该方法为私有方法，只能在类的内部调用 ，不能在类的外部调用self.__private_methods

self

- 类的方法与普通的函数只有一个特别的区别——它们必须有一个额外的第一个参数名称, 按照惯例它的名称是 self
- self代表类的实例，而非类

self 代表的是类的实例，代表当前对象的地址
self.class 则指向类

```
>>> class Test:
    def prt(self):
        print(self)
        print(self.__class__)


>>> t = Test()
>>> t.prt()
<__main__.Test object at 0x0000021D10C1E370>
<class '__main__.Test'>
```

类的方法

在类的内部，使用 def 关键字来定义一个方法，与一般函数定义不同，类方法必须包含参数 self, 且为第一个参数，self 代表的是类的实例

```
>>> class people:
    #定义基本属性
    name = ''
    age = 0
    #定义私有属性,私有属性在类外部无法直接进行访问
    __weight = 0
    #定义构造方法
    def __init__(self,n,a,w):
        self.name = n
        self.age = a
        self.__weight = w
    def speak(self):
        print("%s 说: 我 %d 岁。" %(self.name,self.age))


>>> p = people('runoob',10,30)
>>>
>>> p.speak()
runoob 说: 我 10 岁。
```

继承

派生类的定义

```
class DerivedClassName(BaseClassName1):
        <statement-1>
         . . .
        <statement-N>


class DerivedClassName(modname.BaseClassName):


class DerivedClassName(Base1, Base2, Base3):
        <statement-1>
        . . .
        <statement-N>
```

模块

一个以.py结尾的独立的程序代码的文件

- 一个函数相当于一块积木
- 一个模块可以有很多函数，相当于一盒积木
- 把能够实现某一特定功能的代码放置在一个文件中作为一个模块，方便其他程序和脚本使用，同时避免函数名和变量名冲突

模块导入方法

三种方法

- import module
- from 模块名 import 函数名
- import 模块名 as 新名字



史骏
juns@hfut.edu.cn